Bit Manipulation

1. Introduction to Bit Manipulation [Theory]

Introduction

This article serves as an introduction to some fundamental concepts in binary number manipulation, including binary number conversion, one's complement, and two's complement and many more concepts. These concepts are not only fundamental to understanding how computers work but also crucial for solving a variety of problems in computer science.

Having an understanding of binary numbers is extremely important in the field of computer science as everything stored in a computer is in the form of only 0s and 1s.

Binary Number Conversion

Decimal to Binary Conversion:

By repeatedly dividing a number by 2 and recording the result, decimal values can be transformed into binary.

Example: Converting 13 to its binary equivalent:

- Start with the decimal number 13.
- Divide the number by 2 and record the remainder.
- Repeat the division with the quotient until the number becomes 0.

```
13 / 2 = 6 remainder 1
6 / 2 = 3 remainder 0
3 / 2 = 1 remainder 1
1 / 2 = 0 remainder 1
```

To obtain the binary equivalent of 13, read the remainders from bottom to top: **1101**.

So, the binary equivalent of 13 is 1101.

Binary to Decimal Conversion:

Converting a binary number back to its decimal equivalent involves a reverse process.

Example: Converting 1101 to its decimal equivalent:

- Start from the rightmost bit (least significant bit).
- Each bit is multiplied by 2 raised to the power of its position index.

```
1 * 2^0 = 1
0 * 2^1 = 0
```

```
1 * 2^2 = 4
1 * 2^3 = 8
```

Sum = 1 + 0 + 4 + 8 = **13**.

Hence, the decimal equivalent of the binary number 1101 is 13.

Understanding One's Complement and Two's Complement

One's Complement

The one's complement of a binary number is obtained by flipping all the bits.

Example: The one's complement of 13 (binary 1101):

```
Binary of 13     : 0000 1101
One's Complement : 1111 0010
```

Two's Complement

The two's complement is obtained by taking the one's complement of a number and adding 1.

Example: The two's complement of 13 (binary 1101):

```
One's Complement : 1111 0010
Add 1            : 1111 0011
```

Bitwise Operators

AND Operator (&)

If both corresponding bits are 1, the resulting bit is 1; otherwise, it is 0.

```
13: 1101
 7: 0111
& : 0101 → 5
```

OR Operator (|)

If either corresponding bit is 1, the resulting bit is 1.

```
13: 1101
 7: 0111
|  : 1111 → 15
```

XOR Operator (^)

If bits differ, the result is 1; if the same, result is 0.

```
13: 1101
 7: 0111
^  : 1010 → 10
```

NOT Operator (~)

Flips all bits of the number.

```
5: 0000 0101
~5: 1111 1010 → -6 (in two's complement)
```

Shift Operators

Right Shift (>>): Shifts bits to the right, fills left with 0s.

```
13 >> 1 = 0110 → 6
```

Left Shift (<<): Shifts bits to the left, fills right with 0s.

```
13 << 1 = 11010 → 26
```

Bit Manipulation Tricks and Techniques

1. Swapping Two Numbers Without a Third Variable

```
A = A ^ B
```

```
B = A ^ B        (old A ^ old B) ^ old B = old A
A = A ^ B        (old A ^ old B) ^ old A = old B
```

2. Checking if the i-th Bit is Set

```
(1 << i) & num   → set if result ≠ 0
(num >> i) & 1   → set if result ≠ 0
```

3. Setting the i-th Bit

```
num | (1 << i)
```

4. Clearing the i-th Bit

```
num & ~(1 << i)
```

5. Toggling the i-th Bit

```
num ^ (1 << i)
```

2. Check if the i-th bit is set or not

Given two integers n and i, you need to check whether the i-th bit in the binary representation of n is set (equal to 1). Bit indexing starts from the least significant bit (rightmost) and is 0-based. If the bit is 1, return true, otherwise return false.

Example:
If n = 5, its binary form is 101.
Indexing from right:
0-th bit → 1
1-st bit → 0
2-nd bit → 1

So for i = 0, answer is true.
For i = 1, answer is false.

Approach 1: Brute Force

Algorithm

First convert the given number n into its binary representation.
Store each bit starting from the least significant bit.
Once the binary form is ready, check whether the index i exists.
If i is greater than or equal to the binary length, the bit is not set.
Otherwise, check if the bit at index i is equal to '1'.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    bool checkIthBit(int n,int i) {
        string binary="";
        while(n>0){
            if(n%2==0) binary+='0';
            else binary+='1';
            n/=2;
        }
        if(i>=binary.size()) return false;
        return binary[i]=='1';
    }
};

int main(){
    Solution sol;
    int n=5;
    int i=0;
    cout<<sol.checkIthBit(n,i);
    return 0;
}
```

Complexity Analysis

Time Complexity: O(log n), because the number is converted to binary.
Space Complexity: O(log n), for storing the binary representation.

Approach 2: Optimal Approach (Bit Masking)

Algorithm

Create a mask by left shifting 1 by i positions.
This mask has only the i-th bit set.
Perform a bitwise AND between n and the mask.
If the result is non-zero, the i-th bit is set.
If the result is zero, the i-th bit is not set.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    bool checkIthBit(int n,int i) {
        return (n & (1<<i))!=0;
    }
};

int main(){
    Solution sol;
    int n=5;
    int i=0;
    cout<<sol.checkIthBit(n,i);
    return 0;
}
```

Complexity Analysis

Time Complexity: O(1), only one bitwise operation is used.
Space Complexity: O(1), no extra space is required.

3. Check if a number is odd or not

Given a non-negative integer n, determine whether the number is odd.
A number is considered odd if it is not divisible by 2, meaning when divided by 2 it leaves a remainder.
If the number is odd, return true, otherwise return false.

Example:
If n = 7, dividing by 2 leaves remainder 1, so it is odd.
If n = 10, dividing by 2 leaves no remainder, so it is not odd.

Approach 1

Algorithm
An odd number always leaves a remainder when divided by 2.
Check the remainder of n when divided by 2.
If the remainder is not equal to 0, the number is odd.
Otherwise, the number is even.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    bool isOdd(int n) {
        return (n % 2 != 0);
    }
};

int main() {
    Solution sol;
    int n = 7;
    cout << sol.isOdd(n);
    return 0;
}
```

Complexity Analysis

Time Complexity: O(1), only one modulus operation is performed.
Space Complexity: O(1), no extra space is used.

4. Check if a number is power of 2 or not

Given an integer n, determine whether it is a power of two.
A number is called a power of two if it can be written as 2 raised to some integer power, like 1, 2, 4, 8, 16, and so on.
If n satisfies this condition, return true, otherwise return false.

Example:
If n = 16, it can be written as 2^4, so it is a power of two.
If n = 3, it cannot be written as 2 raised to any integer, so it is not a power of two.

Approach 1

Algorithm

A power of two has exactly one bit set to 1 in its binary representation.
When we subtract 1 from such a number, all bits after that single set bit become 1.
Doing a bitwise AND between the number and one less than the number gives 0 only for powers of two.
Also, n must be greater than 0 because non-positive numbers cannot be powers of two.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    bool isPowerOfTwo(int n) {
        return n>0 && (n&(n-1))==0;    N>0 jaruri
    }
```

```
};

int main() {
    Solution sol;
    int n=16;
    cout<<sol.isPowerOfTwo(n);
    return 0;
}
```

Complexity Analysis
Time Complexity: O(1), only constant-time bitwise operations are used.
Space Complexity: O(1), no extra space is required.

5. Count the number of set bits

Given an integer n, you need to count how many bits are set to 1 in its binary representation.
A set bit means a bit with value 1.
Return the total count of such bits.

Example:
If n = 5, its binary representation is 101. There are two 1s, so the answer is 2.
If n = 15, its binary representation is 1111. There are four 1s, so the answer is 4.

Approach 1: Brute Force

Algorithm
Initialize a counter to store the number of set bits.
While the number is greater than 0:
Check the least significant bit using bitwise AND with 1.
If it is 1, increment the counter.
Right shift the number by 1 to move to the next bit.
Repeat until all bits are checked.
Return the counter.

Code

```cpp
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int countSetBits(int n) {
        int count=0;
        while(n>0){
            if(n&1) count++;
            n>>=1;
        }
        return count;
    }
};

int main(){
    Solution sol;
    int n=5;
    cout<<sol.countSetBits(n);
    return 0;
}
```

Complexity Analysis

Time Complexity: O(log n), each bit of the number is checked once.
Space Complexity: O(1), only a counter variable is used.

Approach 2: Optimal Approach (Brian Kernighan's Algorithm)

Algorithm

Initialize a counter to store the number of set bits.
While the number is greater than 0:
Remove the rightmost set bit by doing n = n & (n - 1).
Increment the counter each time a set bit is removed.
Repeat until the number becomes 0.
Return the counter.

This works because each operation removes exactly one set bit.

Ex: n = 1100

n - 1 = 1011

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int countSetBits(int n) {
        int count=0;
        while(n>0){
            n=n&(n-1);
            count++;
        }
        return count;
    }
};

int main(){
    Solution sol;
    int n=5;
    cout<<sol.countSetBits(n);
    return 0;
}
```

Complexity Analysis

Time Complexity: O(k), where k is the number of set bits in n.
Space Complexity: O(1), no extra space is used.

6. Set the rightmost bit

Given a positive integer n, you need to set the rightmost unset (0) bit in its binary representation to 1 and return the new number.
If all the bits of the number are already set to 1, then the number should remain unchanged.

Example:
If n = 10, its binary form is 1010.
The rightmost unset bit is at position 0. After setting it to 1, the binary becomes 1011, which is 11.

If n = 7, its binary form is 111.
All bits are already set, so the result remains 7.

Approach 1

Algorithm
Add 1 to the number n.
When 1 is added, the rightmost unset bit in n becomes set, and all bits to its right become 0.
Perform a bitwise OR between n and (n + 1).
This ensures the rightmost unset bit is set to 1 while all other bits remain unchanged.
Return the result.

n = 100111

n + 1 = 101000

Code

```
#include <bits/stdc++.h>
using namespace std;

int setRightmostUnsetBit(int n) {
    return n|(n+1);
}

int main() {
    int n=10;
    cout<<setRightmostUnsetBit(n);
```

```
    return 0;
}
```

Complexity Analysis

Time Complexity: O(1), only constant-time bitwise operations are used.
Space Complexity: O(1), no extra space is required.

7. Swap two numbers

Given two integers a and b, swap their values in-place using only two variables.
You are not allowed to use any temporary variable.

Example:
If a = 5 and b = 10, after swapping a becomes 10 and b becomes 5.
If a = -100 and b = -200, after swapping a becomes -200 and b becomes -100.

Approach 1

Algorithm

Use the XOR operator to swap values without extra space.
First, store a ^ b in a.
Then store (a ^ b) ^ b in b, which gives the original value of a.
Finally store (a ^ b) ^ a in a, which gives the original value of b.
This works because XOR of the same numbers becomes 0 and XOR with 0 gives the number itself.

Code

```
#include <bits/stdc++.h>
using namespace std;

void swapXOR(int &a,int &b){
    a=a^b;
    b=a^b;
    a=a^b;
```

```
}

int main(){
    int a=5,b=10;
    swapXOR(a,b);
    cout<<"a = "<<a<<", b = "<<b;
    return 0;
}
```

Complexity Analysis
Time Complexity: O(1), only constant number of operations are performed.
Space Complexity: O(1), no extra space is used.

8. Divide two integers without using multiplication, division and mod operator

Given two integers dividend and divisor, divide them without using multiplication, division, or modulo operators.
The result should be truncated toward zero, meaning the fractional part is removed.
If the result overflows the 32-bit signed integer range, return the maximum or minimum allowed value.

Example:
If dividend = 10 and divisor = 3, the division result is 3.33, which becomes 3 after truncation.
If dividend = 7 and divisor = -3, the division result is -2.33, which becomes -2.

Approach 1: Brute Force

Algorithm
First, determine the sign of the final answer based on the signs of dividend and divisor.
Convert both numbers to their absolute values to simplify calculations.

Repeatedly subtract the divisor from the dividend until the remaining value is smaller than the divisor.
Count how many times the subtraction is possible; this count is the quotient.
Handle overflow cases according to the problem statement.
Apply the correct sign to the final answer and return it.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int divide(int dividend,int divisor){
        if(dividend==INT_MIN && divisor==-1) return INT_MAX;
        bool isPositive=true;
        if((dividend<0 && divisor>0)||(dividend>0 && divisor<0))
            isPositive=false;
        long long n=llabs((long long)dividend);
```

The `function llabs()` in C++ is a standard library function used to calculate the absolute value of a `long long int` number. It is similar to `abs()` and `labs()` but specifically designed for the larger `long long` data type.
`abs()` function is used to calculate the absolute value of a number

```
        long long d=llabs((long long)divisor);
        long long ans=0,sum=0;
        while(sum+d<=n){
            sum+=d;
            ans++;
        }
        if(ans>INT_MAX && isPositive) return INT_MAX;
        if(ans>INT_MAX && !isPositive) return INT_MIN;
        return isPositive?ans:-ans;
    }
};

int main(){
    Solution sol;
    int dividend=10,divisor=3;
    cout<<sol.divide(dividend,divisor);
    return 0;
}
```

Complexity Analysis

Time Complexity: O(dividend), in the worst case when divisor is 1.

Space Complexity: O(1), only constant extra variables are used.

Approach 2: Optimal Approach (Using Bit Shifting)

Algorithm

Determine the sign of the result based on the signs of dividend and divisor.

Convert both numbers to positive values using absolute values.

Instead of subtracting the divisor one by one, subtract the largest possible power-of-two multiple of the divisor.

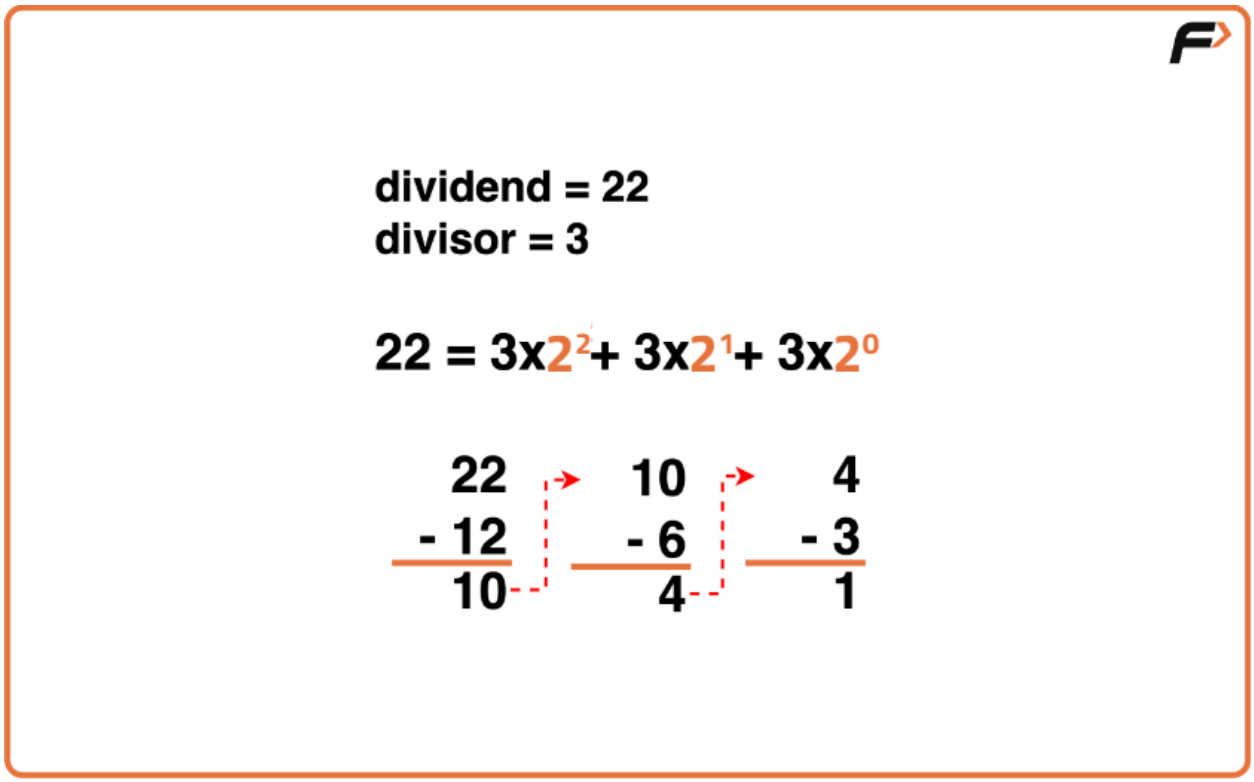
Use left shifting to find the largest multiple of the divisor that is less than or equal to the remaining dividend.

Subtract that value from the dividend and add the corresponding power of two to the answer.

Repeat this process until the dividend becomes smaller than the divisor.

Clamp the result if it exceeds the 32-bit signed integer range.

Apply the correct sign and return the final quotient.



Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int divide(int dividend,int divisor){
        if(dividend==INT_MIN && divisor==-1) return INT_MAX;
        bool isPositive=true;
        if((dividend<0 && divisor>0)||(dividend>0 && divisor<0))
            isPositive=false;
        long long n=llabs((long long)dividend);
        long long d=llabs((long long)divisor);
        long long ans=0;
        while(n>=d){
            long long temp=d;
            long long multiple=1;
            while((temp<<1)<=n){
                temp<<=1;
                multiple<<=1;
            }
            n-=temp;
            ans+=multiple;
        }
        if(ans>INT_MAX && isPositive) return INT_MAX;
        if(ans>INT_MAX && !isPositive) return INT_MIN;
        return isPositive?ans:-ans;
    }
};

int main(){
    Solution sol;
    int dividend=10,divisor=3;
    cout<<sol.divide(dividend,divisor);
    return 0;
}
```

Complexity Analysis

Time Complexity: $O((\log N)^2)$, where N is the absolute value of the dividend.
Space Complexity: O(1), only constant extra variables are used.

9. Count number of bits to be flipped to convert A to B

Given two integers start and goal, you need to find the minimum number of bit flips required to convert start into goal.
A bit flip means changing a bit from 0 to 1 or from 1 to 0 in the binary representation.

Example:
If start = 10 and goal = 7
Binary of 10 is 1010
Binary of 7 is 0111
The bits at three positions are different, so 3 flips are needed.

If start = 3 and goal = 4
Binary of 3 is 011
Binary of 4 is 100
All three bits are different, so 3 flips are needed.

Approach 1

Algorithm

To find differing bits between two numbers, compare them bit by bit.
Use XOR between start and goal.
XOR produces 1 at positions where the bits are different.
Each set bit in the XOR result represents one required flip.
Count the number of set bits in the XOR result to get the answer.

Code

```
#include <bits/stdc++.h>
using namespace std;
```

```
class Solution {
public:
    int minBitsFlip(int start,int goal){
        int num=start^goal;
        int count=0;
        for(int i=0;i<32;i++){
            count+=(num&1);
            num>>=1;
        }
        return count;
    }
};

int main(){
    Solution sol;
    int start=10,goal=7;
    cout<<sol.minBitsFlip(start,goal);
    return 0;
}
```

Complexity Analysis

Time Complexity: O(1), the loop always runs for 32 bits.

Space Complexity: O(1), only constant extra variables are used.

10. Find the number that appears odd number of times

Given an array of integers where every element appears exactly twice except one element that appears only once, find and return that unique element.

The problem guarantees that exactly one such element exists.

Example:
For nums = [1, 2, 2, 4, 3, 1, 4]
All numbers appear twice except 3, so the answer is 3.

For nums = [5]
Only one element exists and it appears once, so the answer is 5.

Approach 1: Brute Force

Algorithm

Use a hashmap to store the frequency of each number.
Traverse the array and for each element, increase its count in the map.
After filling the map, traverse it to find the element whose frequency is exactly 1.
Return that element.
If no such element is found (though the problem guarantees one), return -1.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int singleNumber(vector<int>& nums){
        unordered_map<int,int> mpp;
        for(int i=0;i<nums.size();i++){
            mpp[nums[i]]++;
        }
        for(auto it:mpp){
            if(it.second==1){
                return it.first;
            }
        }
        return -1;
    }
};

int main(){
    vector<int> nums={1,2,2,4,3,1,4};
    Solution sol;
```

```
    cout<<sol.singleNumber(nums);
    return 0;
}
```

Complexity Analysis

Time Complexity: O(N), where N is the size of the array.
Space Complexity: O(N), hashmap stores frequencies of elements.

Approach 2: Optimal Approach (Using XOR)

Algorithm

Initialize a variable XOR to 0.
Traverse the array and XOR each element with this variable.
XOR of two equal numbers is 0, so all numbers appearing twice cancel out.
The number that appears once remains as the final XOR value.
Return this value.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int singleNumber(vector<int>& nums){
        int XOR=0;
        for(int i=0;i<nums.size();i++){
            XOR^=nums[i];
        }
        return XOR;
    }
};

int main(){
    vector<int> nums={1,2,2,4,3,1,4};
    Solution sol;
    cout<<sol.singleNumber(nums);
    return 0;
}
```

Complexity Analysis

Time Complexity: O(N), the array is traversed once.
Space Complexity: O(1), only one variable is used.

11. Power Set | Bit Manipulation

Given an array of integers, generate and print all possible subsets of the array using bitwise operations.
A power set contains every possible combination of elements, including the empty set and the full array.

Example:
If nums = [1, 2, 3], there are 3 elements.
The total number of subsets is $2^3 = 8$.
Each subset can be represented using a binary number from 0 to 7, where each bit tells whether to include an element or not.

For example, binary 101 means include index 0 and index 2 → subset [1, 3].

Approach 1

Algorithm

Let N be the size of the array.
The total number of subsets is 2^n, which can be computed as (1 << n).
Each number from 0 to $(2^n - 1)$ represents one subset.
For each number, check its binary representation.
If the i-th bit is set, include nums[i] in the current subset.
Store each generated subset in the result list.
Return all subsets.

Code

```
#include <bits/stdc++.h>
using namespace std;
```

```
class Solution {
public:
    vector<vector<int>> getPowerSet(vector<int>& nums) {
        int n=nums.size();
        int total=1<<n;
        vector<vector<int>> ans;
        for(int mask=0;mask<total;mask++){
            vector<int> subset;
            for(int i=0;i<n;i++){
                if(mask&(1<<i)){
                    subset.push_back(nums[i]);
                }
            }
            ans.push_back(subset);
        }
        return ans;
    }
};

int main(){
    vector<int> nums={5,7,8};
    Solution sol;
    vector<vector<int>> res=sol.getPowerSet(nums);
    for(auto subset:res){
        cout<<"[ ";
        for(int x:subset) cout<<x<<" ";
        cout<<"]"<<endl;
    }
    return 0;
}
```

Complexity Analysis

Time Complexity: $O(N \times 2^n)$, for each of the 2^n subsets we may scan all N elements.
Space Complexity: $O(N \times 2^n)$, to store all subsets of the power set.

12. Find XOR of numbers from L to R

Given two integers L and R, find the XOR of all numbers in the inclusive range [L, R].
XOR is a bitwise operation where equal bits cancel each other and different bits produce 1.

Example:
If L = 3 and R = 5, the result is 3 ^ 4 ^ 5 = 2.
If L = 1 and R = 3, the result is 1 ^ 2 ^ 3 = 0.

Approach 1: Brute Force

Algorithm
Initialize a variable ans with 0.
Traverse all numbers from L to R.
For each number, take XOR with ans and update ans.
After finishing the loop, ans contains the XOR of all numbers in the range.
Return ans.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int findRangeXOR(int l,int r){
        int ans=0;
        for(int i=l;i<=r;i++){
            ans^=i;
        }
        return ans;
    }
};

int main(){
    Solution sol;
    int l=3,r=5;
```

```
    cout<<sol.findRangeXOR(l,r);
    return 0;
}
```

Complexity Analysis

Time Complexity: O(N), where N = R − L + 1.
Space Complexity: O(1), only one variable is used.

Approach 2: Optimal Approach

Algorithm

The XOR of numbers from 1 to n follows a fixed pattern based on n % 4.
If n % 4 == 0, XOR from 1 to n is n.
If n % 4 == 1, XOR from 1 to n is 1.
If n % 4 == 2, XOR from 1 to n is n + 1.
If n % 4 == 3, XOR from 1 to n is 0.

To find XOR from L to R:
Compute XOR from 1 to R.
Compute XOR from 1 to L − 1.
XOR these two results to cancel common values and get the answer.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
    int XORtillN(int n){
        if(n%4==0) return n;
        if(n%4==1) return 1;
        if(n%4==2) return n+1;
        return 0;
    }
public:
    int findRangeXOR(int l,int r){
        return XORtillN(l-1)^XORtillN(r);
    }
};
```

```
int main(){
    Solution sol;
    int l=3,r=5;
    cout<<sol.findRangeXOR(l,r);
    return 0;
}
```

Complexity Analysis

Time Complexity: O(1), constant time computation.
Space Complexity: O(1), no extra space is used.

13. Find the two numbers appearing odd number of times

Given an array of integers where every element appears exactly twice except for two elements that appear only once, find those two unique numbers.
Return the two numbers in ascending order.

Example:
For nums = [1, 2, 1, 3, 5, 2]
All numbers appear twice except 3 and 5, so the answer is [3, 5].

For nums = [-1, 0]
Both numbers appear once, so the answer is [-1, 0].

Approach 1: Brute Force

Algorithm

Use a hashmap to store the frequency of each element.
Traverse the array and update the frequency of each number.
After that, traverse the hashmap and collect the elements whose frequency is 1.
Sort the result containing the two numbers.
Return the sorted result.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    vector<int> singleNumber(vector<int>& nums){
        unordered_map<int,int> mpp;
        for(int i=0;i<nums.size();i++){
            mpp[nums[i]]++;
        }
        vector<int> ans;
        for(auto it:mpp){
            if(it.second==1){
                ans.push_back(it.first);
            }
        }
        sort(ans.begin(),ans.end());
        return ans;
    }
};

int main(){
    vector<int> nums={1,2,1,3,5,2};
    Solution sol;
    vector<int> ans=sol.singleNumber(nums);
    cout<<ans[0]<<" "<<ans[1];
    return 0;
}
```

Complexity Analysis

Time Complexity: O(N), where N is the size of the array.
Space Complexity: O(N), hashmap is used to store frequencies.

Approach 2: Optimal Approach (Using XOR)

Algorithm

XOR all elements of the array. Duplicate elements cancel out, leaving XOR of the two unique numbers.
Find the rightmost set bit in this XOR result. This bit differs between the two unique numbers.
Traverse the array again and divide numbers into two groups based on this bit.
XOR numbers within each group separately.
Each group will give one unique number.
Sort the two numbers and return them.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    vector<int> singleNumber(vector<int>& nums){
        long x=0;
        for(int i=0;i<nums.size();i++){
            x^=nums[i];
        }
        int rightmost=(x&(x-1))^x;
        int a=0,b=0;
        for(int i=0;i<nums.size();i++){
            if(nums[i]&rightmost) a^=nums[i];
            else b^=nums[i];
        }
        if(a<b) return {a,b};
        return {b,a};
    }
};

int main(){
    vector<int> nums={1,2,1,3,5,2};
    Solution sol;
    vector<int> ans=sol.singleNumber(nums);
    cout<<ans[0]<<" "<<ans[1];
    return 0;
}
```

Complexity Analysis

Time Complexity: O(N), the array is traversed twice.
Space Complexity: O(1), only constant extra variables are used.

14. Prime factors of a number

Given a number n, find all its prime factors.
A prime factor is a prime number that divides n exactly.
A prime number is a natural number greater than 1 that has exactly two factors: 1 and itself.

Example:
If n = 18, its prime factorization is 2 × 3 × 3, so the output is [2, 3, 3].
If n = 25, its prime factorization is 5 × 5, so the output is [5, 5].

Approach 1: Naive Approach (Iterative)

Algorithm

Start checking divisibility from i = 2 up to n.
For each i, if n is divisible by i, then i is a prime factor.
Keep dividing n by i as long as n % i == 0 and store i each time.
Continue this process until i reaches n.
All stored values are the prime factors.

Code

```
#include <bits/stdc++.h>
using namespace std;

vector<int> primeFactor(int n){
    vector<int> ans;
    for(int i=2;i<=n;i++){
        while(n%i==0 && n>0){
            ans.push_back(i);
            n=n/i;
        }
```

```
    }
    return ans;
}

int main(){
    int n=18;
    vector<int> ans=primeFactor(n);
    for(int x:ans){
        cout<<x<<" ";
    }
    return 0;
}
```

Complexity Analysis

Time Complexity: O(n), in the worst case we try all numbers from 2 to n.
Space Complexity: O(1), excluding the space used to store the result.

Approach 2: Expected Approach (Using Square Root)

Algorithm

First, remove all factors of 2 by repeatedly dividing n by 2 and storing 2 each time.
After this step, n becomes odd.
Now check only odd numbers starting from 3 up to √n.
For each i, while n is divisible by i, store i and divide n by i.
Increment i by 2 each time to skip even numbers.
After the loop, if n is greater than 2, then n itself is a prime factor and should be added.

Code

```
#include <bits/stdc++.h>
using namespace std;

vector<int> primeFactors(int n){
    vector<int> factors;

    while(n%2==0){
        factors.push_back(2);
        n=n/2;
    }
```

```
    for(int i=3;i*i<=n;i=i+2){
        while(n%i==0){
            factors.push_back(i);
            n=n/i;
        }
    }

    if(n>2){
        factors.push_back(n);
    }

    return factors;
}

int main(){
    int n=18;
    vector<int> ans=primeFactors(n);
    for(int x:ans){
        cout<<x<<" ";
    }
    return 0;
}
```

Complexity Analysis

Time Complexity: O(√n), because we only iterate up to the square root of n.

Space Complexity: O(1), excluding the space used to store the result.

15. All Divisors of a Number

You are given an integer n. You need to find all the divisors of n and return them in sorted order.

A divisor of a number is a value that divides the number completely without leaving any remainder.

Example:
If n = 6, the numbers that divide 6 exactly are 1, 2, 3, and 6.
If n = 8, the numbers that divide 8 exactly are 1, 2, 4, and 8.

Approach 1: Brute Force

Algorithm

Loop from 1 to n.
For each number i, check if n % i == 0.
If yes, then i is a divisor of n.
Store all such values in a list.
Return the list.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    vector<int> divisors(int n){
        vector<int> ans;
        for(int i=1;i<=n;i++){
            if(n%i==0){
                ans.push_back(i);
            }
        }
        return ans;
    }
};

int main(){
    int n=6;
    Solution sol;
    vector<int> ans=sol.divisors(n);
    for(int x:ans){
        cout<<x<<" ";
    }
    return 0;
```

}

Complexity Analysis

Time Complexity: O(N), since we check all numbers from 1 to n.
Space Complexity: O(√N), as a number can have at most 2√N divisors.

Approach 2: Optimal Approach

Algorithm

Divisors come in pairs. If i divides n, then (n / i) is also a divisor.
Iterate only from 1 to √n.
Whenever i divides n, add i to the list.
If i is not equal to n / i, also add n / i.
After collecting all divisors, sort the list.
Return the sorted list.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    vector<int> divisors(int n){
        vector<int> ans;
        int limit=sqrt(n);
        for(int i=1;i<=limit;i++){
            if(n%i==0){
                ans.push_back(i);
                if(i!=n/i){  //duplicate avoid karna
                    ans.push_back(n/i);
                }
            }
        }
        sort(ans.begin(),ans.end());
        return ans;
    }
};
```

```
int main(){
    int n=6;
    Solution sol;
    vector<int> ans=sol.divisors(n);
    for(int x:ans){
        cout<<x<<" ";
    }
    return 0;
}
```

Complexity Analysis

Time Complexity: O(√N + K log K), where K is the number of divisors (sorting step).
Space Complexity: O(√N), used to store the divisors.

16. Sieve of Eratosthenes

You are given a 2D array queries of size n×2.
Each query [L, R] asks how many prime numbers exist in the range from L to R (inclusive).
You must return an array where each element is the answer for the corresponding query.

A prime number is a number greater than 1 that has exactly two factors: 1 and itself.

Example:
For queries = [[2,5],[4,7]]
The range 2 to 5 has primes 2, 3, 5 → count = 3
The range 4 to 7 has primes 5, 7 → count = 2

Approach 1: Sieve of Eratosthenes + Prefix Sum

Algorithm

First, find the maximum R value among all queries.
Create a boolean array isPrime from 0 to maxR and mark all values as true initially.
Set isPrime[0] and isPrime[1] to false because they are not prime.

Using the Sieve of Eratosthenes, mark all non-prime numbers as false.
Build a prefix sum array primeCount where primeCount[i] stores how many primes exist from 1 to i.
For each query [L, R], the number of primes is primeCount[R] − primeCount[L − 1].
Store the result for every query and return it.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    vector<int> primesInRange(vector<vector<int>>& queries) {
        int maxVal = 0;
        for(auto q : queries){
            maxVal = max(maxVal, q[1]);
        }

        vector<bool> isPrime(maxVal + 1, true);
        if(maxVal >= 0) isPrime[0] = false;
        if(maxVal >= 1) isPrime[1] = false;

        for(int i = 2; i * i <= maxVal; i++){
            if(isPrime[i]){
                for(int j = i * i; j <= maxVal; j += i){
i*i kyoki Koi bhi composite number jiska smallest prime factor i hai,
wo i*i se chhota nahi ho sakta.
                    isPrime[j] = false;
                }
            }
        }

        vector<int> primeCount(maxVal + 1, 0);
        for(int i = 1; i <= maxVal; i++){
            primeCount[i] = primeCount[i - 1];
            if(isPrime[i]) primeCount[i]++;
        }

        vector<int> result;
```

```
        for(auto q : queries){
            int L = q[0];
            int R = q[1];
            if(L == 0)
                result.push_back(primeCount[R]);
            else
                result.push_back(primeCount[R] - primeCount[L - 1]);
        }

        return result;
    }
};

int main(){
    vector<vector<int>> queries = {{2,5},{4,7}};
    Solution sol;
    vector<int> ans = sol.primesInRange(queries);
    for(int x : ans){
        cout << x << " ";
    }
    return 0;
}
```

Complexity Analysis

Time Complexity: O(N log log N), where N is the maximum value in the queries, due to the Sieve of Eratosthenes.

Space Complexity: O(N), for storing the prime array and prefix sum array.

17. Find Prime Factorisation of a Number using Sieve

You are given an integer array queries.
For each number in the array, you need to return its prime factorization in sorted order.
Prime factorization means breaking a number into a product of prime numbers.

Example:
If queries = [2, 3, 4, 5, 6]
2, 3, and 5 are already prime.
4 = 2 × 2
6 = 2 × 3

So the result is [[2], [3], [2, 2], [5], [2, 3]].

Approach 1: Sieve using Smallest Prime Factor (SPF)

Algorithm
Create an array SPF where SPF[i] stores the smallest prime factor of i.
Initially, set all values of SPF as 1.

Use a modified Sieve of Eratosthenes:
Traverse numbers from 2 to MAX_N.
If SPF[i] is still 1, then i is a prime number.
Mark i as the smallest prime factor for all its multiples that are not marked yet.

To find prime factorization of a number n:
Repeatedly take SPF[n], add it to the answer, and divide n by SPF[n].
Continue until n becomes 1.

Apply this process for every number in the queries array.

Code

```
#include <bits/stdc++.h>
using namespace std;

#define MAX_N 100000

vector<int> SPF(MAX_N+1,1);
```

```
class Solution{
private:
    void sieve(){
        for(int i=2;i<=MAX_N;i++){
            if(SPF[i]==1){
                for(int j=i;j<=MAX_N;j+=i){
                    if(SPF[j]==1){
                        SPF[j]=i;
                    }
                }
            }
        }
    }

    vector<int> primeFact(int n){
        vector<int> ans;
        while(n!=1){
            ans.push_back(SPF[n]);
            n=n/SPF[n];
        }
        return ans;
    }

public:
    vector<vector<int>> primeFactors(vector<int>& queries){
        sieve();
        vector<vector<int>> ans;
        for(int i=0;i<queries.size();i++){
            ans.push_back(primeFact(queries[i]));
        }
        return ans;
    }
};

int main(){
    vector<int> queries={2,3,4,5,6};
    Solution sol;
```

```
    vector<vector<int>> ans=sol.primeFactors(queries);
    for(int i=0;i<ans.size();i++){
        for(int x:ans[i]){
            cout<<x<<" ";
        }
        cout<<endl;
    }
    return 0;
}
```

Complexity Analysis

Time Complexity:
O(max_N log log max_N) for building the SPF array using sieve.
O(N log(num)) for factorizing N numbers, where num is the value of a number.

Space Complexity:
O(max_N) for the SPF array.
O(N log(num)) for storing the prime factors of all queries.

🧠 Dry run

MAX_N = 10

i = 2 (prime)

```
SPF[2] = 2
SPF[4] = 2
SPF[6] = 2
SPF[8] = 2
SPF[10] = 2
```

---

i = 3 (prime)

```
SPF[3] = 3
SPF[6] already set (2) → skip
SPF[9] = 3
```

---

i = 4

```
SPF[4] != 1 → skip (composite)
```

---

i = 5 (prime)

```
SPF[5] = 5
SPF[10] already set → skip
```

Final SPF:

```
Index:  2 3 4 5 6 7 8 9 10
SPF:    2 3 2 5 2 7 2 3 2
```

🔹 **Step 3: Prime factorization using SPF**

```
vector<int> primeFact(int n){
    vector<int> ans;
    while(n!=1){
        ans.push_back(SPF[n]);
        n = n / SPF[n];
    }
    return ans;
}
```

🧪 Example: n = 36

```
36 → SPF[36] = 2 → n = 18
18 → SPF[18] = 2 → n = 9
9  → SPF[9]  = 3 → n = 3
3  → SPF[3]  = 3 → n = 1
```

👉 Factors: 2 2 3 3

18. Power (n, x)

You are given a number x and an integer n.
You need to calculate x raised to the power n, written as x^n.
If n is negative, the result should be the reciprocal of x raised to |n|.

Example:
If x = 2 and n = 10, then $2^{10} = 1024$.
If x = 2 and n = -2, then $2^{-2} = 1 / (2^2) = 0.25$.

Approach 1: Brute Force

Algorithm
If n is 0, return 1 because any number raised to 0 is 1.
If n is negative, convert the problem by taking reciprocal of x and making n positive.
Initialize result as 1.
Multiply result with x exactly n times using a loop.
Return the final result.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    double myPow(double x,int n){
        if(n==0) return 1;
        long long exp=n;
        if(exp<0){
            x=1/x;
            exp=-exp;
        }
        double ans=1;
        for(long long i=0;i<exp;i++){
            ans*=x;
        }
        return ans;
    }
```

```
};

int main(){
    Solution sol;
    cout<<sol.myPow(2.0,10)<<endl;
    cout<<sol.myPow(2.0,-2)<<endl;
    return 0;
}
```

Complexity Analysis

Time Complexity: O(n), because multiplication is done n times.
Space Complexity: O(1), no extra space is used.

Approach 2: Optimal Approach (Binary Exponentiation)

Algorithm

Use recursion to reduce the number of multiplications.
If n is 0, return 1.
If n is negative, compute power for -n and return its reciprocal.
If n is even, compute power(x * x, n / 2).
If n is odd, compute x * power(x, n - 1).
This reduces the number of operations significantly.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
private:
    double power(double x,long n){
        if(n==0) return 1;
        if(n==1) return x;
        if(n%2==0){
            return power(x*x,n/2);
        }
        return x*power(x,n-1);
    }
public:
```

```
    double myPow(double x,int n){
        long num=n;
        if(num<0){
            return 1.0/power(x,-num);
        }
        return power(x,num);
    }
};

int main(){
    Solution sol;
    cout<<sol.myPow(2.0,10)<<endl;
    cout<<sol.myPow(2.0,-2)<<endl;
    return 0;
}
```

Complexity Analysis

Time Complexity: O(log n), because the exponent is reduced at each step.

Space Complexity: O(log n), due to recursive call stack.